

CS112: Friday 25 September, 12:15 to 1:20

DRAFT, RECONSTRUCTED FROM COURSE-MAP.YML AND THE TEACHING LOG

Wednesday we wrote `operator<<` and found out why it returns `ostream&` and why it cannot be a method of the class it prints.

Today: go back over the week, then answer the question it kept raising. `Pair` holds two `int`s because the header says `int`, in about six places. So what do you do when you want a pair of `double`s? The answer is one line, and it is the line every class you meet from here on opens with.

PART

1. The week again, from the header down
2. Six places that say `int`
3. One line names the type

Part 1. The week again, from the header down

Rebuild `Pair` on the board from nothing, asking rather than telling: what goes in the header, what goes in the `.cpp`, which members are private, which methods are `const`, and where `operator<<` lives.

Talk to your neighbor. Which of these could you change tomorrow without breaking anyone else's code?

Anything private. Nothing public. Same answer as Monday, arrived at twice.

Part 2. Six places that say `int`

Count them out loud in the header: two members, two constructor parameters, two getter return types.

You now want a `Pair` of `double`s. What do you change?

"All six" is the answer that comes back, and it has a trap in it: some of the `int`s in a real class are counters and indices, which must stay `int`. A

find-and-replace gets those too.

Part 3. One line names the type

```
typedef int Item;
```

Then the class says `Item` everywhere, and changing what a `Pair` holds is this:

```
typedef double Item;
```

`Item` **was never a type of its own**. It is a name for whatever the `typedef` currently points at, which is why nothing else in the class has to move.

Next week the same idea goes one step further: instead of the header choosing the type once, the person using the class chooses it every time.